



ORTA DOĞU TEKNİK ÜNİVERSİTESİ
MIDDLE EAST TECHNICAL UNIVERSITY

CAMPUS RIDE

CNG 352 Term Project

Badenur Turgut

İclal Akman

Miray Murat

PROJECT DESCRIPTION

Campus Ride is a database-driven web application that aims to improve safety and organisation in university transportation by providing an official platform instead of informal ride arrangements. The system is built around a single user account structure, where every participant enters the application as a user and can act as a passenger or a driver depending on their profile information. Users who have a registered driver profile and a driver ID are allowed to offer rides, while all users can request rides as passengers. When a passenger creates a ride request, they specify details such as destination, pickup location, preferred time, passenger count, and route. Drivers can then respond to these requests by proposing a price, and if the passenger accepts the offer, an agreement is formed and the ride starts at the agreed time and route. The database includes entities such as Users, Driver Profiles, Vehicles, Ride Requests, Ride Matches, Locations, Ratings. One of the main challenges of this project is designing a clear and scalable database schema that supports role separation, price negotiation, and ride status tracking while keeping relationships simple and consistent.

DATA REQUIREMENTS

User:

The system stores information about each user including user_ID, email, user_name, phone_number, name, surname, and gender. Each user must be uniquely identified by user_ID, email, and user_name. The User entity is specialized into three roles, namely Admin, Passenger, and Driver. This specialization is overlapping, which means that a user may have more than one role at the same time. For example, a user can be both a Passenger and a Driver if they use the system to request rides and also provide transportation service. A user may also report inappropriate situations related to rides, such as risky driving behavior or any problematic situation during the ride process.

Admin:

The system includes admins to manage and control the platform. Admin is a specialization of User. An admin has permissions stored in the system. These permissions can include updating, deleting, editing, and reviewing posts or system operations. Since an admin may have more than one permission, permissions are represented as a multivalued attribute in the EER diagram. Admins can regulate the system and control platform activities when needed.

Passenger:

The system allows passengers to create ride request posts. Passenger is a specialization of User. A passenger can create multiple posts in the system, while each post is created by one passenger. After a post is created, drivers may respond to that post by making offers. A passenger can receive offers through the posts they create and can select one of the offers to form a ride. A passenger can also rate rides after the ride process.

Driver:

The system allows drivers to respond to ride requests and provide transportation service. Driver is a specialization of User. A driver can make multiple offers for different posts in the system. A driver may also drive multiple rides. Each driver can have one or more cars registered in the system, and these cars can be used during rides. The Driver entity also has a derived rate attribute. This rate value is calculated from the ratings related to the rides driven by the driver. Drivers are one of the core entities that make the ride matching and transportation process possible.

Car:

The system stores information about the cars used by drivers. Car details include plate_number, brand, model, and seat_number. Each car must be uniquely identified by plate_number. A car belongs to one driver, while a driver may have multiple cars registered in the system. Cars are also allocated to rides when a ride is formed.

Post:

The system allows passengers to create ride request posts. Information stored for a post includes post_ID, passenger_number, location, and date_time. Each post must be uniquely identified by post_ID. A passenger can create multiple posts. After a post is created, multiple drivers may respond to that same post by making offers. Therefore, a post can receive more than one offer, but only one of these offers can be selected by the passenger to generate a ride.

Offer:

The system allows drivers to send offers in response to posts. Offer is used to represent the responses of drivers to passenger posts. Each offer is uniquely identified by offer_ID. A driver can make multiple offers, and each offer is made by one driver. A post can receive multiple offers, while each offer belongs to one post. Once an offer is selected, it generates a ride in the system. In this way, offer acts as an important intermediate entity between Post and Ride because it represents the negotiation and selection process before a ride is confirmed.

Ride:

The system stores rides that are formed after an offer is selected. Ride details include ride_ID, start_destination, end_destination, cost, and rating. Each ride must be uniquely identified by ride_ID. A ride is generated from exactly one selected offer. A driver drives the ride, and a car is allocated to that ride. A ride can also be rated by a passenger after the ride process. In this way, the Ride entity represents the final confirmed transportation process between the passenger and the driver.

Data Entry

- Enter the details of a new user in the CampusRide system. (such as a student registered with user_ID: 1, user_name: miray35, name: Miray, surname: Murat, email: e252654@metu.edu.tr, phone_number: 05514404885, gender: female)
- Enter the details of a driver in the system. (such as a driver named Badenur Turgut with user_ID: 2 who is registered in the system and provides ride services.)
- Enter the details of a passenger in the system. (such as a passenger named İclal Akman with user_ID: 3 who uses the system to request transportation.)
- Enter the details of a car registered in the system. (such as a car with plate_number: KZ958, brand: Mercedes, model: c180 Kompresor, seat_number: 5 that belongs to a driver in the system.)
- Enter the details of a ride request post created by a passenger. (such as a post created by passenger İclal Akman with post_ID: 001, passenger_number: 2, location: METU NCC Campus, and date_time: 05.12.2026 14:30.)
- Enter the details of an offer made by a driver for a ride request. (such as driver Badenur Turgut making an offer for post 001 with a proposed transportation service.)
- Enter the details of selecting an offer for a ride request. (such as passenger İclal Akman selecting the offer made by driver Badenur Turgut for the post 001.)
- Enter the details of a ride generated after an offer is accepted. (such as ride_ID: 999 created from the selected offer with start_destination: METU NCC Campus, end_destination: Nicosia City Center, and cost: 1200 TL.)
- Enter the details of a car allocated to a ride. (such as the car with plate_number: KZ958 being allocated to ride 999.)
- Enter the details of a rating given by a passenger to a driver. (such as passenger İclal Akman giving a rating with scale: 5 to driver Badenur Turgut after the ride, with is_anonymous: true.)
- Enter the details of a report submitted in the system. (such as a registered user reporting inappropriate behavior of a driver during a ride.)

Data Delete / Update Operations

- Update/Delete the registered user profile details: This operation can be performed by the registered user who owns the profile. The user can update or delete their personal information stored in the system including name, surname, email, phone_number, gender, and user_name.
- Update/Delete the admin profile details: This operation can only be performed by the admin. Admin can modify or remove admin-related information and permissions maintained in the system.
- Update/Delete passenger information: This operation can be performed by the registered user who has the passenger role. The passenger can update or remove their passenger information connected to their registered account.
- Update/Delete driver information: This operation can be performed by the registered user who has the driver role. The driver can update driver information or remove their driver profile from the system.
- Update/Delete car details in the system: This operation can be performed by the driver who owns the car or by the admin. The information stored for a car including plate_number, brand, model, and seat_number can be modified or removed.
- Update/Delete the ride request post: This operation can be performed by the passenger who created the post or by the admin. The passenger can modify or delete the ride request information including location, passenger_number, and date_time.
- Update/Delete driver offers: This operation can be performed by the driver who created the offer or by the admin. The driver can modify or remove the offer submitted in response to a passenger's ride request.
- Update/Delete ride information: If necessary, this operation can be performed by the admin. Ride details including start_destination, end_destination, and cost stored in the system can be updated or removed.
- Update/Delete rating information: This operation can be performed by the passenger who created the rating or by the admin. The passenger may modify or delete the rating value stored in the system.
- Update/Delete report records: This operation can be performed by the admin. Admin can update or remove report records submitted by users in order to manage the system and maintain platform integrity.

Data Queries

- List all users registered in the CampusRide system.
- List all admins in the system together with their permissions.
- List all passengers registered in the system.
- List all drivers in the system together with their current rate values.
- List all cars registered in the system together with the drivers who own them.
- List all ride request posts created by passengers.
- List all posts created by the passenger Miray Murat.
- List all offers made by drivers for ride request posts.
- List all offers received for a specific post.
- List the details of generated rides, including the passenger, driver, allocated car, start destination, end destination, cost, and rating.
- List all posts with the number of offers they received.
- List all rides driven by the driver Badenur Turgut.
- List all reports submitted by passengers for rides.
- Identify the total number of users, passengers, drivers, posts, offers, rides, and reports in the system.
- List all posts together with the number of offers they received.
- List the average rating of each driver based on completed rides.
- List the top three drivers according to their average ride rating.
- List all rides whose cost is greater than 1000 TL.
- List all rides created from offers made for posts created by İclal Akman.
- Update the phone number of Miray Murat.
- Update the user_name of badenur48 to bTugut.
- Delete a report record submitted for a specific ride.
- Delete an offer if it has not generated any ride yet.
- Update driver rates according to the average rating of their completed rides. Only drivers whose rate value is different from the new average are updated.
- Insert a new offer only if the driver has enough car seats and has not already made an offer for the same post.
- Delete offers that were not accepted for a specific post.
- Apply a 10% discount to expensive rides with low ratings.

Questions

Q1) The proposed CampusRide system connects passengers and drivers within the university community to make transportation easier for students. Although the system improves mobility, it may also introduce some public safety risks if not properly managed. Passenger safety is an important concern because rides are provided by other students. To reduce this risk, the system stores verified user information such as user_ID to ensure that users belong to the university community. The rating and reporting mechanisms also help identify unsafe drivers and encourage responsible behavior. In addition, road safety must be considered because drivers are responsible for following traffic regulations. From a social perspective, the system helps students who do not own a car access transportation more easily. Environmentally, ride sharing can reduce the number of vehicles around campus and decrease traffic congestion. Economically, it provides a more affordable transportation option for students compared to traditional taxi services. Moreover, this would also create a safe and self-sustaining source of income for the students.

Q2) The CampusRide system must address several ethical and legal considerations to protect users and ensure responsible software development. One major ethical issue is user privacy because the system stores personal data such as user_ID, email and phone_number. According to the IEEE and ACM Codes of Ethics, computing professionals must protect user data and prevent unauthorized access. Therefore, the system should implement secure authentication and proper data protection mechanisms. Another ethical concern is fairness within the rating system since passengers evaluate drivers after rides. The system must ensure that ratings are transparent and cannot be manipulated. In terms of legal considerations, the platform must comply with data protection regulations and institutional policies when handling personal information. Intellectual property is also important because the system design and software implementation must respect licensing rules.

EER Diagram

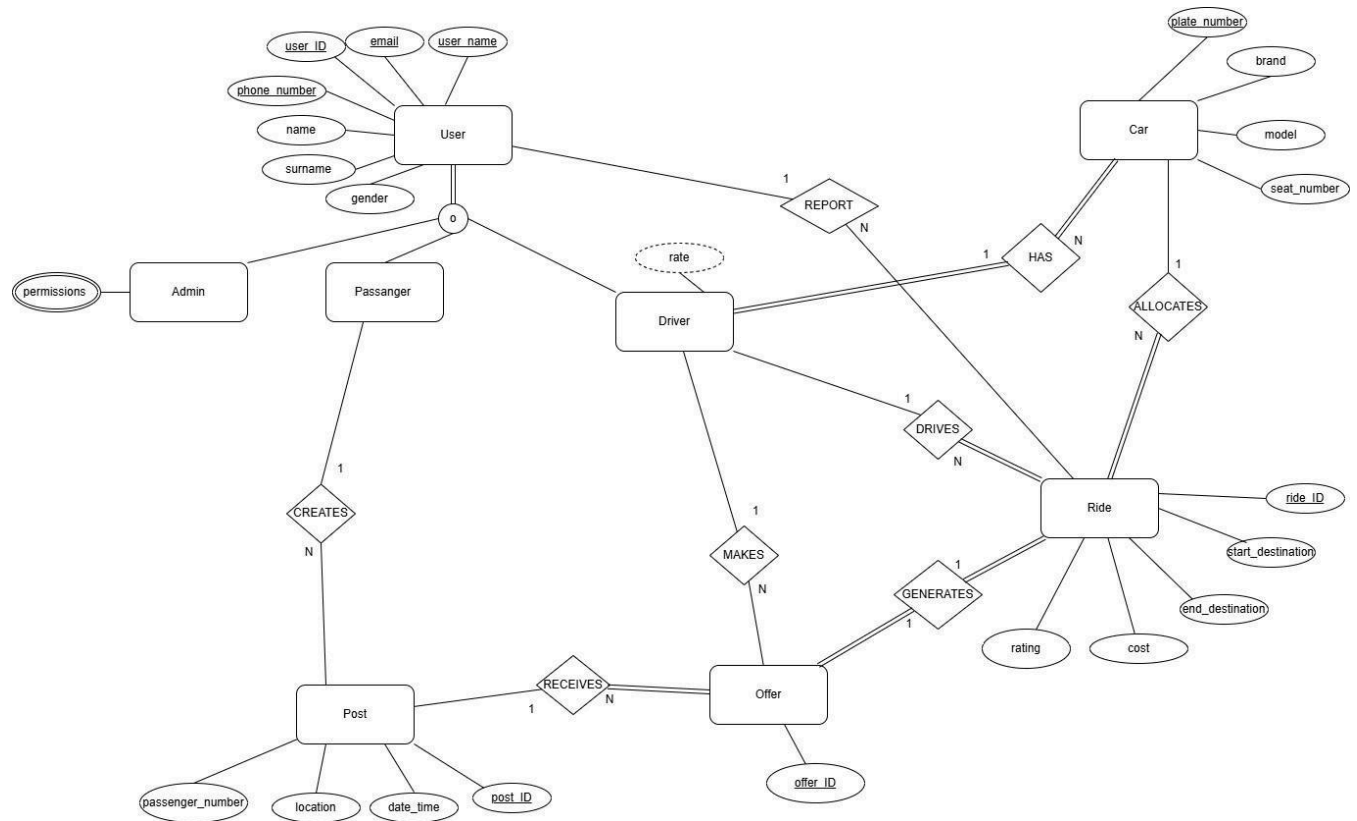


Figure 1: Campus Ride

Assumptions

- Each user is uniquely identified by user_ID, email, and user_name. These attributes ensure that each user account in the system belongs to a unique individual within the university community.
- A ride is created only after a passenger selects one of the offers submitted by drivers. Each ride is associated with exactly one selected offer.
- Each driver must have a registered car in the system before providing ride services. A car belongs to one driver, while a driver may use that car for multiple rides.
- After completing a ride, passengers can rate the driver using the rating system. Ratings are stored with a scale value and may optionally be anonymous.

- The admin has full authority and permissions within the system. Admins can access, modify, and manage all system data, including users, posts, offers, rides, cars, ratings, and reports. Admins also have the authority to alter or remove inappropriate posts, delete dishonest offers, manage user accounts, review reports submitted by users, and maintain the overall integrity and security of the system.
- Passenger number is not the ID of the passenger who posted that message; it's an attribute indicating how many people will be on the ride.

(E) ER MAPPING & FUNCTIONAL DEPENDENCIES

A. USER

<u>user_ID</u>	email	user_name	phone_number	name	surname	gender
----------------	-------	-----------	--------------	------	---------	--------

user_ID --> email, user_name, phone_number, name, surname, gender

email --> user_ID

user_name --> user_ID

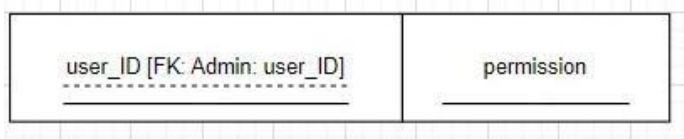
phone_number --> user_ID

B. ADMIN



user_ID -> (no additional attributes)

C. ADMINPERMISSION



user_ID, permission --> (no additional attributes)

D. PASSENGER

<u>user_ID [FK: User: user_ID]</u>

user_ID -> (no additional attributes)

E. DRIVER

<u>user_ID [FK: User: user_ID]</u>	rate
------------------------------------	------

user_ID -> rate

F. CAR

<u>plate_number</u>	brand	model	seat_number	driver_ID [FK: Driver: user_ID]
---------------------	-------	-------	-------------	---------------------------------

plate_number -> brand, model, seat_number, driver_ID

G. POST

<u>post_ID</u>	passenger_number	location	date_time	passenger_ID [FK: Passenger: user_ID]
----------------	------------------	----------	-----------	---------------------------------------

post_ID -> passenger_number, location, date_time, passenger_ID

H. OFFER

<u>offer_ID</u>	post_ID [FK: Post: post_ID]	driver_ID [FK: Driver: user_ID]
-----------------	-----------------------------	---------------------------------

offer_ID -> post_ID, driver_ID

I. RIDE

<u>ride_ID</u>	cost	start_destination	end_destination	rating	offer_ID [FK: Offer: offer_ID]	driver_ID [FK: Driver: user_ID]	plate_number [FK: Car: plate_number]	user_ID [FK: User: user_ID]
----------------	------	-------------------	-----------------	--------	---	--	---	--------------------------------------

ride_ID -> cost, start_destination, end_destination, rating, offer_ID, driver_ID,
plate_number, user_ID
offer_ID -> ride_ID (Each accepted offer generates only one ride)
plate_number -> driver_ID (Each car belongs to one driver)

Mapping Choices:

- Each regular entity type was mapped into a separate relation with its own primary key and attributes.
- The specialization between User and its subtypes was mapped by using the inherited user_ID in each subtype relation.
- Since Registered_User and Guest are disjoint subtypes of User, both relations use user_ID as their primary key and also as a foreign key referencing User.
- Since Admin, Passenger, and Driver are overlapping subtypes of Registered_User, separate relations were created for each subtype. Their user_ID attribute is both a primary key and a foreign key referencing Registered_User.
- 1:N relationships (HAS, CREATES, MAKES, RECEIVES, DRIVES, ALLOCATES, REPORT) were mapped by placing the primary key of the 1-side as a foreign key in the relation on the N-side.
- 1:1 relationships (SELECTS, GENERATES) were mapped by placing the foreign key in the relation with total participation.
- Their primary keys are composite keys formed from the foreign keys of the participating entities.
- The multivalued attribute permissions were mapped as a separate relation: ADMINPERMISSION (user_ID, permission).

FINAL STEPS OF FUNCTIONAL DEPENDENCIES & NORMALIZATION

1. USER

Primary Key: user_ID

Functional Dependencies:

user_ID -> email, user_name, phone_number, name, surname, gender

email -> user_ID

user_name -> user_ID

phone_number -> user_ID

Normalization:

This table is in 1NF because all attributes are atomic. It is in 2NF because the primary key is a single attribute, and all non-key attributes are fully dependent on user_ID. It is in 3NF because all attributes depend directly on the primary key and not through another non-key attribute.

2. ADMIN

Primary Key: user_ID

Functional Dependency:

user_ID -> (no additional attributes)

Normalization:

This table is in 1NF, 2NF, and 3NF because it contains only the inherited key and no non-key attributes.

3. ADMINPERMISSION

Primary Key: (user_ID, permission)

Functional Dependency:

user_ID, permission -> (no additional attributes)

Normalization:

Considering that one admin can have multiple permissions, and each permission entry belongs to one admin.

Therefore, this table is in 1NF because each permission is stored as a separate atomic value. It is in 2NF because there is no non-key attribute that depends on part of the composite key. It is in 3NF because there is no transitive dependency.

4. PASSENGER

Primary Key: user_ID

Functional Dependency:

user_ID -> (no additional attributes)

Normalization:

This table is in 1NF, 2NF, and 3NF because it contains only the inherited key and no non-key attributes.

5. DRIVER

Primary Key: driver_ID

Functional Dependency:

driver_ID -> rate

Normalization:

This table is in 1NF because the attributes are atomic. It is in 2NF because the primary key is a single attribute, and the rate fully depends on driver_ID. It is in 3NF because the rate depends directly on the primary key and not transitively.

6. CAR

Primary Key: plate_number

Functional Dependency:

plate_number -> brand, model, seat_number, driver_ID

Normalization:

This table is in 1NF because the values are atomic. It is in 2NF because the key is a single attribute and all non-key attributes depend fully on plate_number. It is in 3NF because there is no transitive dependency; all attributes directly describe the car identified by plate_number.

7. POST

Primary Key: post_ID

Functional Dependency:

post_ID -> passenger_number, location, date_time, passenger_ID

Normalization:

This table is in 1NF because each column contains atomic values. It is in 2NF because the key is

a single attribute and all non-key attributes depend fully on post_ID. It is in 3NF because all attributes depend directly on the primary key.

8. OFFER

Primary Key: offer_ID

Functional Dependency:

offer_ID -> post_ID, driver_ID

Normalization:

This table is in 1NF because all values are atomic. It is in 2NF because the primary key is a single attribute and all non-key attributes depend fully on offer_ID. It is in 3NF because all attributes depend directly on the primary key.

9. RIDE

Primary Key: ride_ID

Functional Dependencies:

ride_ID -> start_destination, end_destination, cost, offer_ID, rating, driver_ID, plate_number, user_ID

offer_ID -> ride_ID (Each accepted offer generates only one ride)

plate_number -> driver_ID (Each car belongs to one driver)

Normalization:

This table is in 1NF because all columns contain atomic values. It is in 2NF because the primary key is a single attribute, and all non-key attributes are fully dependent on ride_ID. However, it's not 3NF because in the Ride table, plate_number -> driver_ID, which is not a key in this table. Therefore, due to transitive dependency, it is not 3NF.

RIDE TABLE AFTER NORMALIZATION:

<u>ride_ID</u>	start_destination	end_destination	cost	offer_ID [FK: Offer: offer_ID]	rating	plate_number [FK: Car: plate_number]
----------------	-------------------	-----------------	------	--------------------------------	--------	--------------------------------------

ride_ID -> start_destination, end_destination, cost, offer_ID, rating, plate_number,

plate_number -> driver_ID

This dependency already belongs to the CAR table; there is no need to make it a new table.

Short Overall Normalisation Summary

1NF:

- All tables are in 1NF because:
- each column contains atomic values,
- each row is uniquely identified by a primary key,
- repeating groups are removed,
- Multivalued attributes, such as admin permissions, are separated into their own table.

2NF:

- All tables are in 2NF because:
- tables with single-attribute primary keys have all non-key attributes fully dependent on the whole key,
- tables with composite keys, such as ADMINPERMISSION, have non-key attributes dependent on the entire composite key,
- Removing many-to-many relationship tables, such as ALTER and GETRATED, has no non-key attributes, so they do not violate 2NF.

3NF:

- All tables are in 3NF because:
- non-key attributes depend directly on the primary key,
- there are no transitive dependencies,
- relationship tables either have no non-key attributes or their non-key attributes depend directly on the full key.

BCNF:

- All tables are in BCNF because every determinant in each relation is a candidate key or a superkey.
- Tables with composite keys, such as ADMINPERMISSION, have all non-key attributes dependent on the entire composite key.
- Relationship tables with non-key attributes have those attributes dependent only on the full composite key.
- There are no determinants that are not candidate keys or superkeys.
- The RIDE relation also satisfies BCNF if the business rule stating that one accepted offer_ID creates only one ride_ID is enforced during implementation.

DENORMALIZATION

The table REPORT, ride_ID and user_ID both will all be foreign keys from the RIDE and Passenger tables. It's better to retrieve user_ID from the PASSENGER table rather than the USER or PASSENGER table; the passenger_ID, which is a passenger in the POST table, will report the ride. Since the passenger does not need to report the ride some of the ratings will be NULL variable therefore, new report table is created for efficiency.

REPORT



Since ride_ID and user_ID are foreign keys and at the same time a composite primary key.

Functional Dependency:

ride_ID and user_ID -> (no additional attributes)

Therefore, there is no redundant and transitive dependency.

The table REPORT follows 3NF/BCNF principles; however, a controlled denormalisation was applied to the table Report by including user_ID. This optimises the retrieval of user-specific feedback without doing complex 4-table joins (Report -> Ride -> Offer -> Post). This trade-off significantly improves query performance while maintaining data integrity.

UPDATED DESIGN PROCESS

In the updated EER design, the previous Guest and Registered User structure was removed, and the model now contains only the User entity, which is directly specialized into Admin, Passenger, and Driver with overlapping participation. The network_ID attribute was also removed from the conceptual design because session-related information is not kept as a main data requirement anymore. In addition, the separate Rating entity and the RATE / GET_RATED relationships were removed; instead, rating is stored as an attribute of Ride, while Driver has a derived rate attribute calculated from ride ratings. The previous SELECTS relationship between Passenger and Offer was also removed, since an accepted offer is now represented through the GENERATES relationship between Offer and Ride. Overall, these changes simplify the design, remove unnecessary relationship overlaps, and make the EER model more consistent with the final relational schema.

SQL QUERIES

PostgreSQL was used for the SQL queries and physical database implementation because it is a reliable relational database management system that supports the main concepts required in our CampusRide project. Since our design includes primary keys, foreign keys, unique constraints, check constraints, and relationships between several tables, PostgreSQL provides a suitable environment to implement these structures clearly. It also supports referential integrity, which helps us maintain consistency between related tables such as User, Passenger, Driver, Post, Offer, Ride, Car, and Report. Another reason for choosing PostgreSQL is that it allows us to test the actual database definition, data insertion, update, delete, and selection queries in a practical way using an Admin. Therefore, PostgreSQL was appropriate for converting our final EER design and relational schema into a real working physical database.

The tables created are as follows:

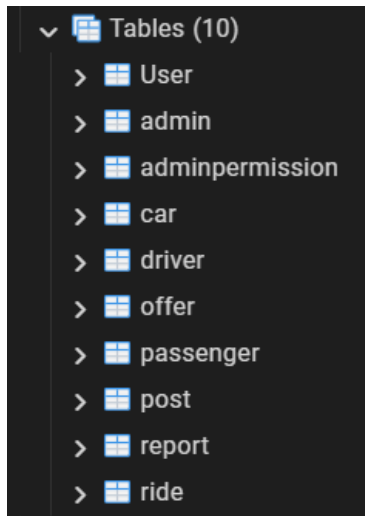


Figure 2: Tables

Inserted Data Tables:

User:

Data Output Messages Notifications							
Showing rows: 1 to 8 Page No: 1							
	user_id [PK] integer	email character varying (100)	user_name character varying (50)	phone_number character varying (20)	name character varying (50)	surname character varying (50)	gender character varying (20)
1	1	e252654@metu.edu.tr	miray35	05514404885	Miray	Murat	Female
2	2	e252673@metu.edu.tr	badenur48	05324567891	Badenur	Turgut	Female
3	3	e258460@metu.edu.tr	iclal26	05431234567	İclal	Akman	Female
4	4	e252701@metu.edu.tr	efe01	05551234567	Efe	Demir	Male
5	5	e252702@metu.edu.tr	zeynep02	05339876543	Zeynep	Kaya	Female
6	6	e252704@metu.edu.tr	mert03	05445556677	Mert	Yılmaz	Male
7	7	e252706@metu.edu.tr	deniz04	05321112233	Deniz	Şahin	Male
8	8	e252707@metu.edu.tr	elif05	05553334455	Elif	Arslan	Female

Figure 3: User Table

Admin:

	user_id [PK] integer 
1	1
2	2
3	3

Figure 4:Admin Table

Admin Permissions:

	user_id [PK] integer 	permission [PK] character varying (50) 
1	1	update
2	1	delete
3	1	edit
4	1	review
5	2	update
6	2	delete
7	2	edit
8	2	review
9	3	update
10	3	delete
11	3	edit
12	3	review

Figure 5:Admin Permission Table

Passenger:

	user_id [PK] integer 
1	1
2	3
3	4
4	5
5	8

Figure 6:Passenger Table

Driver:

	user_id [PK] integer	rate numeric (3,2)
1	2	5.00
2	4	4.00
3	6	4.00
4	7	2.00

Figure 7: Driver Table

Car:

	plate_number [PK] character varying (20)	brand character varying (50)	model character varying (50)	seat_number integer	driver_id integer
1	33 ABC 123	Mercedes	C180 Kompressor	5	2
2	34 EFE 456	Renault	Clio	4	4
3	06 MRT 789	Fiat	Egea	5	6
4	35 DNZ 246	Toyota	Corolla	5	7

Figure 8: Car Table

Post:

	post_id [PK] character varying (20)	passenger_number integer	location character varying (100)	date_time timestamp without time zone	passenger_id integer
1	P001	2	METU NCC Dormitory	2026-05-12 14:30:00	3
2	P002	1	METU NCC Library	2026-05-13 09:00:00	1
3	P003	3	Kalkanlı Gate	2026-05-14 17:45:00	5
4	P004	1	Engineering Faculty	2026-05-15 11:15:00	4
5	P005	2	Main Cafeteria	2026-05-16 18:00:00	8

Figure 9: Post Table

Offer:

	offer_id [PK] character varying (20)	post_id character varying (20)	driver_id integer
1	O001	P001	2
2	O002	P001	4
3	O003	P002	6
4	O004	P003	7
5	O005	P004	2
6	O006	P005	4
7	O007	P003	6

Figure 10: Offer Table

Ride:

	ride_id [PK] character varying (20)	start_destination character varying (100)	end_destination character varying (100)	cost numeric (10,2)	rating integer	offer_id character varying (20)	plate_number character varying (20)
1	R001	METU NCC Dormitory	Nicosia City Center	1200.00	5	O001	33 ABC 123
2	R002	METU NCC Library	Lefkoşa Bus Terminal	900.00	4	O003	06 MRT 789
3	R003	Kalkanlı Gate	Girne City Center	1500.00	2	O004	35 DNZ 246
4	R004	Engineering Faculty	Güzelyurt Center	750.00	5	O005	33 ABC 123
5	R005	Main Cafeteria	Nicosia City Center	1100.00	4	O006	34 EFE 456

Figure 11: Ride Table

Report:

	ride_id [PK] character varying (20)	user_id [PK] integer
1	R003	5
2	R002	1

Figure 12: Report Table

Queries:

1. List all rides driven by Badenur Turgut.

```
34 -- List all rides driven by Badenur Turgut.
35 SELECT
36     r.ride_ID,
37     r.start_destination,
38     r.end_destination,
39     r.cost,
40     r.rating,
41     c.plate_number,
42     c.brand,
43     c.model
44 FROM ride r
45 JOIN offer o ON r.offer_ID = o.offer_ID
46 JOIN "User" u ON o.driver_ID = u.user_ID
47 JOIN car c ON r.plate_number = c.plate_number
48 WHERE u.name = 'Badenur'
49        AND u.surname = 'Turgut'
50 ORDER BY r.ride_ID;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	ride_id character varying (20)	start_destination character varying (100)	end_destination character varying (100)	cost numeric (10,2)	rating integer	plate_number character varying (20)	brand character varying (50)	model character varying (50)
1	R001	METU NCC Dormitory	Nicosia City Center	1200.00	5	33 ABC 123	Mercedes	C180 Kompressor
2	R004	Engineering Faculty	Güzelyurt Center	750.00	5	33 ABC 123	Mercedes	C180 Kompressor

2. List the average rating of each driver based on completed rides.

```
52 -- List the average rating of each driver based on completed rides.
53 SELECT
54     u.user_ID,
55     u.name AS driver_name,
56     u.surname AS driver_surname,
57     ROUND(AVG(r.rating)::NUMERIC, 2) AS average_rating
58 FROM driver d
59 JOIN "User" u ON d.user_ID = u.user_ID
60 JOIN offer o ON d.user_ID = o.driver_ID
61 JOIN ride r ON o.offer_ID = r.offer_ID
62 WHERE r.rating IS NOT NULL
63 GROUP BY u.user_ID, u.name, u.surname
64 ORDER BY average_rating DESC;
```

Data Output Messages Notifications

	user_id [PK] integer	driver_name character varying (50)	driver_surname character varying (50)	average_rating numeric
1	2	Badenur	Turgut	5.00
2	4	Efe	Demir	4.00
3	6	Mert	Yılmaz	4.00
4	7	Deniz	Şahin	2.00

3. List all rides created from offers made for posts created by İclal Akman.

```
65 -- List all rides created from offers made for posts created by İclal Akman.
66 SELECT
67     r.ride_ID,
68     r.start_destination,
69     r.end_destination,
70     r.cost,
71     r.rating,
72     driver_user.name || ' ' || driver_user.surname AS driver
73 FROM ride r
74 JOIN offer o ON r.offer_ID = o.offer_ID
75 JOIN post p ON o.post_ID = p.post_ID
76 JOIN "User" passenger ON p.passenger_ID = passenger.user_ID
77 JOIN "User" driver_user ON o.driver_ID = driver_user.user_ID
78 WHERE passenger.name = 'İclal'
79     AND passenger.surname = 'Akman'
80 ORDER BY r.ride_ID;
```

Data Output Messages Notifications

Showing rows: 1 to 1

	ride_id [PK] character varying (20)	start_destination character varying (100)	end_destination character varying (100)	cost numeric (10,2)	rating integer	driver text
1	R001	METU NCC Dormitory	Nicosia City Center	1200.00	5	Badenur Turgut

4. List all posts with the number of offers they received

```
107 -- List all posts with the number of offers they received.
108 SELECT
109     p.post_ID AS Post_ID,
110     p.location AS Post_Location,
111     p.date_time AS Request_Date_Time,
112     COUNT(o.offer_ID) AS Number_Of_Offers
113 FROM
114     post p
115 JOIN
116     offer o ON p.post_ID = o.post_ID
117 GROUP BY
118     p.post_ID, p.location, p.date_time
119 ORDER BY
120     Number_Of_Offers DESC;
```

Data Output Messages Notifications

	post_id [PK] character varying (20)	post_location character varying (100)	request_date_time timestamp without time zone	number_of_offers bigint
1	P001	METU NCC Dormitory	2026-05-12 14:30:00	2
2	P003	Kalkanlı Gate	2026-05-14 17:45:00	2
3	P002	METU NCC Library	2026-05-13 09:00:00	1
4	P004	Engineering Faculty	2026-05-15 11:15:00	1
5	P005	Main Cafeteria	2026-05-16 18:00:00	1

5. List all rides whose cost is greater than 1000 TL

```
123 -- List all rides whose cost is greater than 1000 TL.
124 SELECT
125     ride_ID AS Ride_ID,
126     start_destination AS Start_Destination,
127     end_destination AS End_Destination,
128     cost AS Cost,
129     rating AS Rating
130 FROM
131     ride
132 WHERE
133     cost > 1000
134 ORDER BY
135     cost DESC;
```

Data Output Messages Notifications

	ride_id [PK] character varying (20)	start_destination character varying (100)	end_destination character varying (100)	cost numeric (10,2)	rating integer
1	R003	Kalkanlı Gate	Girne City Center	1500.00	2
2	R001	METU NCC Dormitory	Nicosia City Center	1200.00	5
3	R005	Main Cafeteria	Nicosia City Center	1100.00	4

6. Insert a new offer

Query History

```
93 -- Update driver rates according to the average rating of their completed rides.
94 -- Only drivers whose rate value is different from the new average are updated.
95 UPDATE Driver d
96 SET rate = sub.avg_rating
97 FROM (
98     SELECT
99         o.driver_ID,
100         ROUND(AVG(r.rating)::NUMERIC, 2) AS avg_rating
101     FROM Offer o
102     JOIN Ride r ON r.offer_ID = o.offer_ID
103     WHERE r.rating IS NOT NULL
104     GROUP BY o.driver_ID
105 ) AS sub
106 WHERE d.user_ID = sub.driver_ID
107 AND d.rate IS DISTINCT FROM sub.avg_rating
108 RETURNING d.user_ID, d.rate;
```

Data Output Messages Notifications

Showing rows: 1 to 8

	offer_id [PK] character varying (20)	post_id character varying (20)	driver_id integer
1	0001	P001	2
2	0003	P002	6
3	0004	P003	7
4	0005	P004	2
5	0006	P005	4
6	0007	P003	6
7	0008	P005	7
8	0002	P001	4

7. Update ride cost based on rating

```
136 -- Apply a 10% discount to expensive rides with low ratings.
137 UPDATE Ride
138 SET cost = ROUND((cost * 0.90)::NUMERIC, 2)
139 WHERE cost > 1000
140 AND rating < 4
141 RETURNING ride_ID, start_destination, end_destination, cost, rating;
142
143
```

Data Output Messages Notifications

Showing rows: 1 to 1

	ride_id [PK] character varying (20)	start_destination character varying (100)	end_destination character varying (100)	cost numeric (10,2)	rating integer
1	R003	Kalkanlı Gate	Girne City Center	1350.00	2

8. Delete not accepted offers

```
126 -- Delete offers that were not accepted for a specific post.
127 DELETE FROM Offer o
128 WHERE o.post_ID = 'P001'
129       AND NOT EXISTS (
130         SELECT 1
131         FROM Ride r
132         WHERE r.offer_ID = o.offer_ID
133       )
134 RETURNING offer_ID, post_ID, driver_ID;
135
136 -- Apply a 10% discount to expensive rides with low ratings
```

Data Output Messages Notifications

Showing row

	offer_id [PK] character varying (20)	post_id character varying (20)	driver_id integer
1	O002	P001	4

WORKLOAD ANALYSIS AND INDEX SELECTION

In this project, PostgreSQL was used for the physical database implementation of the CampusRide system. PostgreSQL was selected because it supports the main relational database concepts required in our project, such as primary keys, foreign keys, unique constraints, check constraints, cascading operations, joins, aggregate functions, data manipulation operations, and indexing mechanisms.

The CampusRide application is a database-driven ride-sharing system designed for a university environment. Therefore, the workload of the application includes both query operations and data manipulation operations. Users interact with the system by creating ride request posts, making offers, accepting offers, generating rides, rating completed rides, and reporting inappropriate ride situations. Because of this, the database must support frequent search, join, insert, update, and delete operations efficiently.

Workload of the Application

The workload of the CampusRide application is mainly read-heavy. This means that SELECT queries are expected to be called more frequently than update or delete operations. In a real system, users usually browse ride request posts, check offers, view ride details, list available drivers, and see rating information more often than they update or delete data.

However, insert and update operations are also important because passengers create new posts, drivers create offers, rides are generated after an offer is accepted, and ratings are updated after completed rides.

The most frequently used query operations in the system are expected to be:

Operation	Description	Expected Frequency
List ride request posts	Users view available ride request posts.	Very frequent
List offers for a post	Passengers check offers received for their posts.	Very frequent
List rides of a driver	Used to see completed rides of a specific driver.	Frequent
List rides of a passenger	Used to see rides created from a passenger's posts.	Frequent
Calculate driver average rating	Used to rank or evaluate drivers.	Frequent
List rides above a certain cost	Used for filtering ride prices.	Sometimes frequent
Count offers for each post	Used to analyze post activity.	Frequent
List reports	Admins review reported rides.	Less frequent

The most frequent data manipulation operations are expected to be:

Operation Type	Description	Expected Frequency
INSERT	A passenger creates a new post.	Very frequent
INSERT	A driver creates an offer for a post.	Very frequent
INSERT	A ride is generated after an offer is accepted.	Frequent
UPDATE	A ride rating is updated after the ride is completed.	Frequent
UPDATE	Driver rate is recalculated according to ride ratings.	Frequent
UPDATE	User profile information is updated.	Less frequent
DELETE	Rejected offers are deleted if they did not generate a ride.	Less frequent
DELETE	Reviewed report records are deleted by admin.	Less frequent

Approximate Number of Records in Tables

In the current implementation, a small amount of sample data was inserted into the database for testing and demonstration purposes. The current approximate number of records in each table is as follows:

Table Name	Current Approximate Number of Records
User	8
Admin	3
AdminPermission	12
Passenger	5
Driver	4
Car	4
Post	5
Offer	7
Ride	5
Report	2

Although the current database contains a small test dataset, the number of records would increase significantly in a real university environment. For example, during an academic semester, the expected approximate number of records can be estimated as follows:

Table Name	Expected Approximate Number of Records in Real Usage
User	2,000 - 5,000
Admin	5 - 20
AdminPermission	20 - 100
Passenger	1,500 - 4,000
Driver	200 - 800
Car	200 - 1,000
Post	5,000 - 15,000
Offer	10,000 - 30,000
Ride	3,000 - 10,000
Report	100 - 1,000

According to these estimates, the largest tables are expected to be Post, Offer, and Ride. This is because passengers can create many posts, each post can receive multiple offers, and accepted offers generate rides. Therefore, indexes should mainly focus on these tables.

Index Selection

PostgreSQL automatically creates indexes for primary keys and unique constraints. Therefore, attributes such as user_ID, post_ID, offer_ID, ride_ID, plate_number, email, user_name, and phone_number already have indexes because they are primary keys or unique attributes.

However, PostgreSQL does not automatically create indexes for all foreign key columns. Since our workload contains many join operations using foreign keys, additional indexes should be created on frequently used foreign key attributes and frequently filtered columns.

The following indexes were selected according to the workload of the CampusRide application.

Index Name	Table / Column(s)	Purpose
idx_post_passenger_id	Post(passenger_ID)	Find posts created by a specific passenger.
idx_post_date_time	Post(date_time)	Find posts according to date and time.
idx_offer_post_id	Offer(post_ID)	List offers received by a specific post.
idx_offer_driver_id	Offer(driver_ID)	List offers made by a specific driver.
idx_offer_post_driver_unique	Offer(post_ID, driver_ID)	Prevent the same driver from making duplicate offers for the same post.
idx_ride_cost	Ride(cost)	Filter rides according to cost.
idx_ride_rating	Ride(rating)	Support rating-based ride and driver evaluation queries.
idx_ride_cost_rating	Ride(cost, rating)	Support operations where cost and rating are used together.
idx_car_driver_id	Car(driver_ID)	Find cars owned by a specific driver.
idx_report_user_id	Report(user_ID)	Find reports submitted by a specific user.
idx_user_name_surname	User(name, surname)	Search users by name and surname in sample queries.

Index Implementation Script

```
-- PostgreSQL Indexes for CampusRide Workload

-- Index for finding posts created by a specific passenger.
CREATE INDEX IF NOT EXISTS idx_post_passenger_id
ON Post(passenger_ID);

-- Index for finding posts according to date and time.
CREATE INDEX IF NOT EXISTS idx_post_date_time
ON Post(date_time);

-- Index for listing offers received by a specific post.
CREATE INDEX IF NOT EXISTS idx_offer_post_id
ON Offer(post_ID);

-- Index for listing offers made by a specific driver.
CREATE INDEX IF NOT EXISTS idx_offer_driver_id
ON Offer(driver_ID);
```

```

-- Unique index to prevent the same driver from making more than one offer for the same post.
CREATE UNIQUE INDEX IF NOT EXISTS idx_offer_post_driver_unique
ON Offer(post_ID, driver_ID);

-- Index for filtering rides according to cost.
CREATE INDEX IF NOT EXISTS idx_ride_cost
ON Ride(cost);

-- Index for filtering rides according to rating.
CREATE INDEX IF NOT EXISTS idx_ride_rating
ON Ride(rating);

-- Composite index for update and filtering operations using both cost and rating.
CREATE INDEX IF NOT EXISTS idx_ride_cost_rating
ON Ride(cost, rating);

-- Index for finding cars owned by a specific driver.
CREATE INDEX IF NOT EXISTS idx_car_driver_id
ON Car(driver_ID);

-- Index for finding reports submitted by a specific user.
CREATE INDEX IF NOT EXISTS idx_report_user_id
ON Report(user_ID);

-- Index for searching users by name and surname.
CREATE INDEX IF NOT EXISTS idx_user_name_surname
ON "User"(name, surname);

```

The selected indexes were chosen according to the expected workload of the CampusRide application. Since the system frequently performs search, join, filtering, and update operations on tables such as Post, Offer, Ride, Car, Report, and "User", indexes were mainly created on frequently used foreign key attributes and filtering columns. The indexes on Post (passenger_ID) and Post (date_time) improve the performance of listing posts created by a specific passenger and finding posts according to date and time. The indexes on Offer (post_ID) and Offer (driver_ID) are important because the Offer table connects passengers' posts with drivers, and the system frequently needs to list offers received for a post or offers made by a driver. In addition, the unique composite index on Offer (post_ID, driver_ID) both improves the duplicate-offer checking process and protects data integrity by preventing the same driver from making more than one offer for the same post. The indexes on Ride(cost) and Ride(rating) support cost-based and rating-based filtering, while the composite index on Ride (cost, rating) helps update operations where both conditions are used together, such as applying discounts to expensive rides with low ratings. The index on Car (driver_ID) helps the system quickly find cars owned by a specific driver, especially when checking seat availability before an offer is made. The index on Report (user_ID) improves the retrieval of reports submitted by a specific user. Finally, the index on "User"(name, surname) supports the current sample queries that search users by their names and surnames, such as listing rides driven by Badenur Turgut or rides related to İclal Akman. Overall, indexes were not created for every attribute because unnecessary indexes may slow down insert, update, and delete operations. Instead, the selected indexes provide a balanced design by improving the performance of frequent queries while avoiding

unnecessary over-indexing.

CHANGES MADE AFTER STEP 4

After Step 4, the CampusRide database schema was updated according to the requirements of the final web application. These changes were not applied as separate ALTER TABLE commands in the final submission. Instead, the final CreateTable.sql file was updated directly, so the physical database is created from the beginning with the application-supported schema.

The first change was made in the User relation. In the earlier version, the User relation stored user_ID, email, user_name, phone_number, name, surname, and gender. In the final application, users can sign up and sign in through the web interface. Therefore, a new attribute named password_hash was added to the User relation. The application does not store plain text passwords. Instead, the password entered by the user is converted into a hashed value and stored in password_hash. This improves security because real passwords are not directly visible in the database.

The updated User relation is:

User(user_ID, email, user_name, phone_number, name, surname, gender, password_hash)

The second change was made in the Post relation. In the previous design, the location attribute represented the pickup location. However, in the final application, a passenger must specify both where they want to be picked up and where they want to go. Therefore, the destination attribute was added to the Post relation. This makes each ride request more complete and allows drivers to evaluate the route before making an offer.

The updated Post relation is:

Post(post_ID, passenger_number, location, destination, date_time, passenger_ID)

The third change was made in the Offer relation. In the final system flow, a passenger creates a ride request post, and drivers respond to this post by proposing a price and selecting one of their registered cars. Therefore, the proposed ride cost and selected car should be stored in the Offer relation. For this reason, cost and plate_number were added to Offer. A UNIQUE(post_ID, driver_ID) constraint was also added to prevent the same driver from sending more than one offer for the same post.

The updated Offer relation is:

Offer(offer_ID, post_ID, driver_ID, cost, plate_number)

This relation includes the following important constraints:

UNIQUE(post_ID, driver_ID)

CHECK(cost >= 0)

The fourth change was made in the Report relation. The rating mechanism and the report mechanism are different from each other. Ride.rating is used only to rate the driver after the ride time has started, and it is used to update the derived Driver.rate value. The Report relation is used for problem reporting. In the final schema, Report was extended with report_ID, report_text, report_type, status, and created_at attributes. This allows users to submit both ride-specific reports and general platform issue reports. A ride-specific report has a ride_ID value, while a general report may have a NULL ride_ID.

The updated Report relation is:

Report(report_ID, ride_ID, user_ID, report_text, report_type, status, created_at)

These changes make the database compatible with the final web application. They also make the application flow more realistic: passengers create posts with pickup and destination information, drivers make offers by entering price and selecting a car, passengers accept one offer, and the system automatically creates a ride from the accepted offer.

```
CREATE TABLE "User" (  
    user_ID SERIAL PRIMARY KEY,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    user_name VARCHAR(50) NOT NULL UNIQUE,  
    phone_number VARCHAR(20) NOT NULL UNIQUE,  
    name VARCHAR(50) NOT NULL,  
    surname VARCHAR(50) NOT NULL,  
    gender VARCHAR(20) NOT NULL,  
    password_hash VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE Offer (  
    offer_ID VARCHAR(20) PRIMARY KEY,  
    post_ID VARCHAR(20) NOT NULL,  
    driver_ID INT NOT NULL,  
    UNIQUE (post_ID, driver_ID),  
    FOREIGN KEY (post_ID) REFERENCES Post(post_ID)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    FOREIGN KEY (driver_ID) REFERENCES Driver(user_ID)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```
CREATE TABLE Post (  
  post_ID VARCHAR(20) PRIMARY KEY,  
  passenger_number INT NOT NULL,  
  location VARCHAR(100) NOT NULL,  
  destination VARCHAR(100) NOT NULL,  
  date_time TIMESTAMP NOT NULL,  
  passenger_ID INT NOT NULL,  
  CHECK (passenger_number > 0),  
  FOREIGN KEY (passenger_ID) REFERENCES Passenger(user_ID)  
    ON UPDATE CASCADE  
    ON DELETE RESTRICT  
);
```

```
CREATE TABLE report (  
  report_ID SERIAL PRIMARY KEY,  
  ride_ID VARCHAR(20),  
  user_ID INT NOT NULL,  
  report_text TEXT NOT NULL,  
  report_type VARCHAR(20) NOT NULL DEFAULT 'Ride',  
  status VARCHAR(20) NOT NULL DEFAULT 'Open',  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  CHECK (report_type IN ('Ride', 'General')),  
  CHECK (ride_ID IS NOT NULL OR report_type = 'General'),  
  
  FOREIGN KEY (ride_ID) REFERENCES Ride(ride_ID)  
    ON UPDATE CASCADE  
    ON DELETE CASCADE,  
  FOREIGN KEY (user_ID) REFERENCES "User"(user_ID)  
    ON UPDATE CASCADE  
    ON DELETE CASCADE  
);
```

APPLICATION IMPLEMENTATION

1) Programming Language and Technologies Used:

The CampusRide application was implemented as a web-based database interface using Python. Python was selected because it allows fast development of database-driven applications and provides simple libraries for both web interface development and PostgreSQL database access. The main framework used in the application is Flask. Flask was used to create web routes, handle user requests, process form inputs, manage sessions, and display different pages according to the role of the logged-in user. The PostgreSQL database was accessed through the psycopg2 library. This library allows the Python application to connect to PostgreSQL and execute SQL selection and manipulation queries.

The main technologies and libraries used in the implementation are:

- Python: Main programming language of the application.
- Flask: Web framework used to create the user interface and application routes.
- psycopg2: PostgreSQL adapter used to connect the application to the database.
- Werkzeug Security: Used for password hashing and password verification.
- PostgreSQL: Relational database management system used for the physical database.
- HTML and CSS: Used inside Flask templates to create the web interface.
- Visual Studio Code: Used as the development environment.

The important libraries imported in the application are:

```
from flask import Flask, request, redirect, url_for, session, render_template_string, flash
from werkzeug.security import generate_password_hash, check_password_hash
import psycopg2
import psycopg2.extras
```

2) Database Connection:

The application connects to the local PostgreSQL database named CampusRide. A separate connection function is used whenever the application needs to read or modify database records. The database password is not shown in the report for security reasons.

```
DB_NAME = "CampusRide"
DB_USER = "postgres"
DB_PASSWORD = "*****"
DB_HOST = "localhost"
DB_PORT = "5432"
def get_connection():
    return psycopg2.connect(
        dbname=DB_NAME,
        user=DB_USER,
        password=DB_PASSWORD,
        host=DB_HOST,
```

```
port=DB_PORT,  
)
```

This function returns a PostgreSQL connection object. The application then uses this connection to execute SELECT, INSERT, UPDATE, and DELETE statements.

3) Application Structure:

The application follows a role-based structure. Each user signs in through the same login page, but the available menu options change according to the roles stored in the database. The User relation is specialized into Admin, Passenger, and Driver. Therefore, after login, the application checks whether the current user exists in the Admin, Passenger, and Driver tables.

The main interface options provided by the application are:

- Sign Up
- Sign In
- View Profile
- Create Ride Request Post
- Make Offer
- View Received Offers
- Accept Offer
- View Rides
- Rate Ride
- Report Ride
- Report General Issue
- Add Car
- Admin Panel
- Logout

Although the assignment describes the interface as a list of alternatives, this application provides these alternatives through web navigation links and buttons. The Logout option works as the quit option because it ends the user session.

4) Role-Based Interface:

The application displays different options according to the user type:

Passenger users can:

- Create ride request posts.
- View offers received for their own posts.
- Accept an offer and create a ride.
- View their rides.
- Rate completed rides.

- Report problematic rides.

Driver users can:

- View ride request posts.
- Make offers for posts.
- View their submitted offers.
- Add car information.
- View rides driven by them.

Admin users can:

- View registered users.
- View admins and their permissions.
- View drivers and driver rates.
- View reports.
- Delete reviewed report records.
- Refresh driver rates.
- Apply discount operations to selected rides.

5) Error Handling:

The application includes error handling for invalid or incomplete operations. For example, during registration, if the user selects Driver but does not enter car information, the system does not complete the registration. If the user enters a password shorter than 8 characters, the system rejects the registration. If a driver tries to send an offer without a suitable car, the offer is not inserted. If a passenger tries to accept an offer that does not belong to their own post, the application redirects the user and does not modify the database.

These checks prevent invalid data from being inserted into the PostgreSQL database and help maintain data integrity.

APPLICATION AND SCREENSHOTS

The CampusRide web application provides a user-friendly interface for accessing and modifying the PostgreSQL database. Instead of using a console menu, the application presents alternatives through navigation buttons and forms. Each page performs a specific database query or modification.

The application satisfies the interface requirements because it provides more than five alternatives to the user. These alternatives include Sign Up, Sign In, Profile, Create Post, Make Offer, Received Offers, Accept Offer, Rides, Rate Ride, Report Issue, Add Car, Admin Panel, and Logout. The Logout option works as the quit option.

1. Sign Up Transaction

The Sign Up page allows a new user to register as Passenger, Driver, or both. If the user selects Driver, the car information fields become visible and required. This transaction inserts data into User, Passenger, Driver, and Car tables depending on the selected role.

Database operation type:

INSERT Tables used:

User, Passenger, Driver, Car

The screenshot shows the CampusRide sign-up interface. On the left, a dark blue card contains the CampusRide logo, the text "Create your CampusRide account.", and instructions: "Choose your role, enter your profile information, and start using the platform with the correct passenger or driver access." Below this are three buttons: "Passenger posts", "Driver offers", and "Ride tracking". On the right, a white card titled "Sign up" prompts the user to "Choose an account type to continue." It contains form fields for Email (e000000@metu.edu.tr), Username (Create a username), Phone Number (05xxxxxxxx), Gender (Choose), Name (First name, Last name), Surname (Last name), Password (At least 8 characters), and Account Type (Choose user type). A "Create Account" button is at the bottom.

This screenshot shows the same CampusRide sign-up interface, but with the "Passenger posts" button selected. The "Account Type" dropdown menu is now set to "Passenger". The "Gender" dropdown is also set to "Choose". The "Create Account" button remains at the bottom of the form.

This screenshot shows the Sign Up page when Passenger is selected as the account type. Since the user selected Passenger, car information fields are not displayed. The form collects the common User attributes such as email, username, phone number, name, surname, gender, and password. After submission, the application inserts the user into the User table and also inserts the same user_ID into the Passenger table.

The screenshot shows the CampusRide Sign Up page. The header includes the CampusRide logo and the tagline "Smart campus ride-sharing platform". The main heading is "Sign up". Below this, there's a section titled "Create your CampusRide account." with a subtext: "Choose your role, enter your profile information, and start using the platform with the correct passenger or driver access." There are three buttons: "Passenger posts", "Driver offers", and "Ride tracking". The "Passenger posts" button is selected. The form fields are as follows:

- Email:** e000000@metu.edu.tr
- Username:** Create a username
- Phone Number:** 05xxxxxxxx
- Gender:** Choose (dropdown menu)
- Name:** First name, Last name
- Surname:** Last name
- Password:** At least 8 characters
- Account Type:** Driver (dropdown menu)

Below the form, there's a section titled "Car Information" with the note "Required only for driver accounts." The fields are:

- Plate Number:** 35 ABC 123
- Brand:** Toyota
- Model:** Corolla
- Seat Number:** 4

A "Create Account" button is at the bottom of the form.

This screenshot shows the Sign Up page when Driver is selected as the account type. In this case, the application displays additional car information fields such as plate number, brand, model, and seat number. After submission, the application inserts records into User, Driver, and Car tables.

This screenshot demonstrates how the interface changes dynamically according to the selected user role.

```
@app.route("/signup", methods=["GET", "POST"])
def signup():
    if request.method == "POST":
        role = request.form.get("role", "")
        password = request.form.get("password", "")

        if len(password) < MIN_PASSWORD_LENGTH:
            flash("Password must be at least 8 characters long.")
            return redirect(url_for("signup"))

        plate = request.form.get("plate_number", "").strip()
        brand = request.form.get("brand", "").strip()
        model = request.form.get("model", "").strip()
        seats = request.form.get("seat_number", "").strip()

        if role in ["driver", "both"] and (not plate or not brand or not model or not seats):
            flash("Driver registration requires complete car information.")
            return redirect(url_for("signup"))
```

```

conn = get_connection()
try:
    cur = conn.cursor()
    cur.execute(
        'INSERT INTO "User"
(email,user_name,phone_number,name,surname,gender,password_hash) VALUES
(%s,%s,%s,%s,%s,%s,%s) RETURNING user_ID;',
        (
            request.form["email"].strip(),
            request.form["user_name"].strip(),
            request.form["phone_number"].strip(),
            request.form["name"].strip(),
            request.form["surname"].strip(),
            request.form["gender"],
            generate_password_hash(password),
        ),
    )
    new_id = cur.fetchone()[0]
    created_parts = ["account"]

    if role in ["passenger", "both"]:
        cur.execute("INSERT INTO passenger(user_ID) VALUES(%s);", (new_id,))
        created_parts.append("passenger profile")

    if role in ["driver", "both"]:
        cur.execute("INSERT INTO driver(user_ID, rate) VALUES(%s,0.00);", (new_id,))
        cur.execute(
            "INSERT INTO car(plate_number,brand,model,seat_number,driver_ID)
VALUES(%s,%s,%s,%s,%s);",
            (plate, brand, model, int(seats), new_id),
        )
        created_parts.extend(["driver profile", "car record"])

    conn.commit()
    cur.close()
    flash("Account created successfully with " + " ".join(created_parts) + ".")
    return redirect(url_for("signin"))

except Exception as err:
    conn.rollback()
    flash(f"Registration failed. No changes were saved. Details: {err}")

finally:
    conn.close()

content = """
<div class="authgrid">
  <div class="authside">
    <div class="featureicon">_ fi </div>
    <h2>Create your CampusRide account.</h2>

```

<p>Choose your role, enter your profile information, and start using the platform with the correct passenger or driver access.</p>

<div class="actions">

Passenger posts

Driver offers

Ride tracking

</div>

</div>

<div class="card">

<h2>Sign up</h2>

<p id="role-note" class="helper">Choose an account type to continue.</p>

<form method="POST">

<div class="grid">

<div>

<label>Email</label>

<input type="email" name="email" placeholder="e000000@metu.edu.tr" required>

</div>

<div>

<label>Username</label>

<input name="user_name" placeholder="Create a username" required>

</div>

<div>

<label>Phone Number</label>

<input name="phone_number" placeholder="05xxxxxxxx" required>

</div>

<div>

<label>Gender</label>

<select name="gender" required>

<option value="">Choose</option>

<option>Female</option>

<option>Male</option>

</select>

</div>

<div>

<label>Name</label>

<input name="name" placeholder="First name" required>

</div>

<div>

<label>Surname</label>

<input name="surname" placeholder="Last name" required>

</div>

<div>

<label>Password</label>

```

        <input type="password" name="password" placeholder="At least 8 characters"
minlength="8" required>
        <p class="helper">Password must be at least 8 characters long.</p>
    </div>

    <div>
        <label>Account Type</label>
        <select id="role" name="role" onchange="toggleCar()" required>
            <option value="">Choose user type</option>
            <option value="passenger">Passenger</option>
            <option value="driver">Driver</option>
            <option value="both">Passenger and Driver</option>
        </select>
    </div>
</div>

<div id="carbox" class="hidden">
    <div class="carhead">
        <div class="icon"> 🚗 ⬆️ </div>
        <div>
            <h3 style="margin:0">Car Information</h3>
            <p class="helper" style="margin:2px 0 0">Required only for driver accounts.</p>
        </div>
    </div>

    <div class="grid">
        <div>
            <label>Plate Number</label>
            <input name="plate_number" placeholder="35 ABC 123">
        </div>

        <div>
            <label>Brand</label>
            <input name="brand" placeholder="Toyota">
        </div>

        <div>
            <label>Model</label>
            <input name="model" placeholder="Corolla">
        </div>

        <div>
            <label>Seat Number</label>
            <input type="number" min="1" name="seat_number" placeholder="4">
        </div>
    </div>

    <button class="primary">Create Account</button>
</form>

```

```
</div>
</div>
****
return layout("Sign Up", content)
```

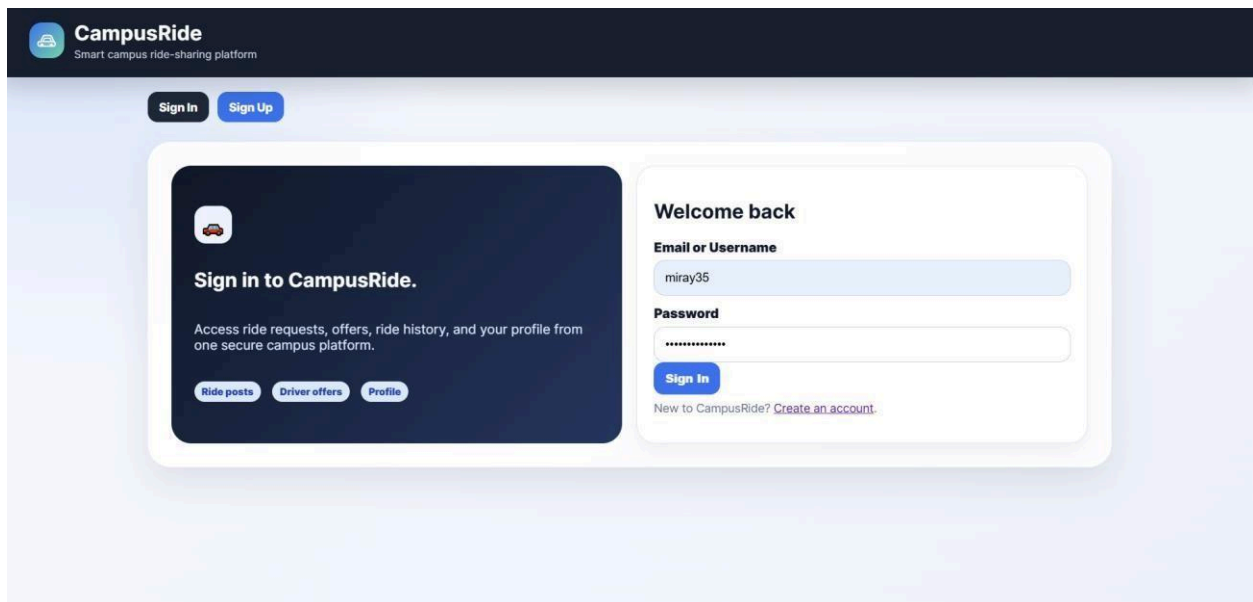
2. Sign In Transaction

The Sign In page allows an existing user to log into the application. The user enters either email or username and password. The application reads the User table and verifies the password using password_hash.

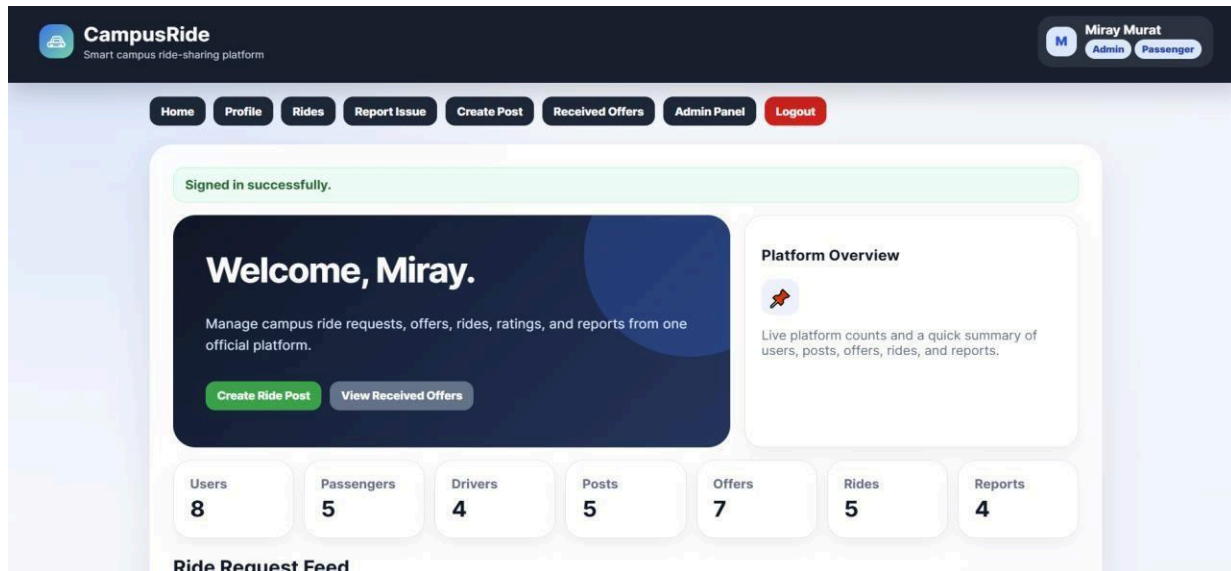
Database operation type: SELECT

Table used:

User



This screenshot shows the Sign In page. The user can log in by entering either email or username and password. The application checks the User table and verifies the entered password using the stored password_hash value. If the credentials are correct, the user session starts and the application displays role-based menu options.



This screenshot shows the home page after login. The page lists available ride request posts with pickup location, destination, date-time, passenger count, and offer count. Drivers can use this page to select a post and make an offer. Passengers can use this page to see their own posts and delete posts that have not generated a ride yet.

```
@app.route("/signin", methods=["GET", "POST"])
def signin():
    if request.method == "POST":
        ident = request.form.get("username_or_email", "").strip()
        password = request.form.get("password", "")
        user = query_one('SELECT user_ID AS user_id, password_hash FROM "User" WHERE
user_name=%s OR email=%s;', (ident, ident))
        if user and user["password_hash"] and check_password_hash(user["password_hash"], password):
            session["user_id"] = user["user_id"]
            flash("Signed in successfully.")
            return redirect(url_for("home"))
            flash("Invalid sign-in information.")
        content = ""
        <div class="authgrid">
            <div class="authside">
                <div class="featureicon"> fi _.</div>
                <h2>Sign in to CampusRide.</h2>
                <p>Access ride requests, offers, ride history, and your profile from one secure campus
platform.</p>
                <div class="actions"><span class="pill">Ride posts</span><span class="pill">Driver
offers</span><span class="pill">Profile</span></div>
            </div>
            <div class="card">
                <h2>Welcome back</h2>
                <form method="POST">
                    <label>Email or Username</label>
```

```

        <input name="username_or_email" placeholder="Enter your email or username" required>
        <label>Password</label>
        <input type="password" name="password" placeholder="Enter your password"
autocomplete="current-password" required>
        <button class="primary">Sign In</button>
    </form>
    <p class="helper">New to CampusRide? <a href="/signup">Create an account</a>.</p>
</div>
</div>
""""
return layout("Sign In", content)

```

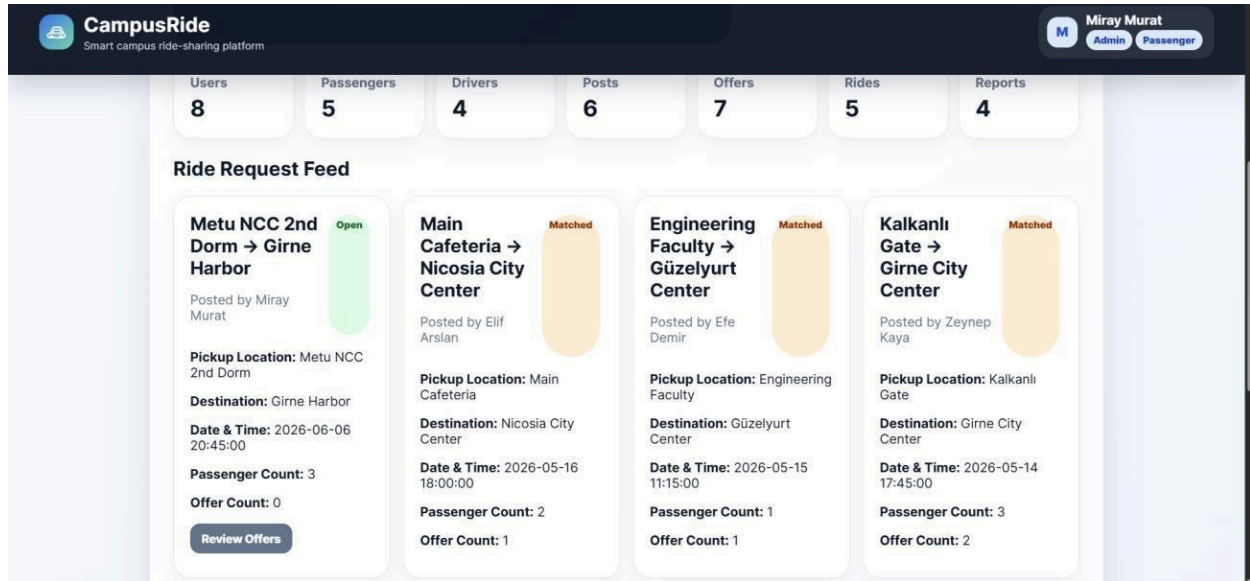
3. Create Ride Request Post Transaction

The Create Post page is available to passengers. The passenger enters pickup location, destination, date-time, and passenger count. After submitting the form, a new row is inserted into the Post table.

Database operation type: INSERT

Table used:

Post



This screenshot shows the Create Ride Request page. A passenger enters pickup location, destination, preferred date-time, and passenger count. When the form is submitted, the application inserts a new row into the Post table. This transaction demonstrates an INSERT operation performed through the user interface.

4. Make Offer Transaction

The Make Offer page is available to drivers. The driver selects a ride request post, enters the proposed price, and selects one of their registered cars. This information is inserted into the Offer table. The application also prevents the same driver from sending more than one offer for the same post.

Database operation type: INSERT and SELECT

Tables used:

Offer, Post, Car, Driver

CampusRide
Smart campus ride-sharing platform

Badenur Turgut
Admin Driver

Make Offer

Choose the car you will use for this ride and enter your proposed price.

Metu NCC 2nd Dorm → Girne Harbor

Passenger: Miray Murat

Pickup Location: Metu NCC 2nd Dorm

Destination: Girne Harbor

Date: 2026-06-06 20:45:00

Passenger Count: 3

Offer Price

Example: 850

Car Used for This Offer

33 ABC 123 - Mercedes C180 Kompressor (5 seats)

Send Offer

This screenshot shows the Make Offer page for drivers. The driver can see the passenger's route information and passenger count. The driver then enters the proposed price and selects one of their registered cars for the ride. After submission, the application inserts a new row into the Offer table with cost and plate_number information.

CampusRide
Smart campus ride-sharing platform

Badenur Turgut
Admin Driver

Home Profile Rides Report Issue My Offers Add Car Admin Panel Logout

Offer sent successfully. Offer ID: 0008.

My Offers

Offer Id	Post Id	Pickup	Destination	Date Time	Passenger	Offer Price	Selected Car	Status	Ride Id
O008	P006	Metu NCC 2nd Dorm	Girne Harbor	2026-06-06 20:45:00	Miray Murat	1000.00	33 ABC 123	Waiting	
O005	P004	Engineering Faculty	Güzelyurt Center	2026-05-15 11:15:00	Efe Demir	750.00	33 ABC 123	Accepted	R004
O001	P001	METU NCC Dormitory	Nicosia City Center	2026-05-12 14:30:00	İclal Akman	1200.00	33 ABC 123	Accepted	R001
O001	P001	METU NCC Dormitory	Nicosia City Center	2026-05-12 14:30:00	İclal Akman	1200.00	33 ABC 123	Accepted	R001

This screenshot shows the offers submitted by a driver. Each offer displays the related post, pickup location, destination, date-time, passenger name, proposed price, selected car, and status. The status can be Waiting, Accepted, or Rejected. If another driver's offer is accepted for the same post, the current driver's offer becomes Rejected.

Data Output						Messages	Notifications
	offer_id [PK] character varying (20)	post_id character varying (20)	driver_id integer	cost numeric (10,2)	plate_number character varying (20)		
1	O001	P001	2	1200.00	33 ABC 123		
2	O002	P001	4	1000.00	34 EFE 456		
3	O003	P002	6	900.00	06 MRT 789		
4	O004	P003	7	1500.00	35 DNZ 246		
5	O005	P004	2	750.00	33 ABC 123		
6	O006	P005	4	1100.00	34 EFE 456		
7	O007	P003	6	1400.00	06 MRT 789		
8	O008	P006	2	1000.00	33 ABC 123		

5. Received Offers and Accept Offer Transaction

The Received Offers page is available to passengers. The passenger can compare offers according to driver name, driver rate, proposed price, selected car, and status. If the passenger accepts one offer, the application automatically creates a Ride row. The ride uses pickup and destination information from Post and cost and plate_number information from Offer.

Database operation type: SELECT and INSERT

Tables used:

Post, Offer, Driver, Car, Ride

CampusRide Smart campus ride-sharing platform									
M Miray Murat Admin Passenger Home Profile Rides Report Issue Create Post Received Offers Admin Panel Logout									
Received Offers									
Post	Route	Date	Offer	Driver	Rate	Price	Car	Status	Action
P006	Metu NCC 2nd Dorm → Girne Harbor	2026-06-06 20:45:00	O008	Badenur Turgut	5.00	1000.00	33 ABC 123 - Mercedes C180 Kompressor	Waiting	Accept Offer
P002	METU NCC Library → Lefkoşa Bus Terminal	2026-05-13 09:00:00	O003	Mert Yılmaz	4.00	900.00	06 MRT 789 - Fiat Egea	Accepted	-

This screenshot shows the offers received by a passenger for their own posts. The passenger can compare drivers according to driver name, driver rate, proposed price, selected car, and offer status. This page performs SELECT operations using Post, Offer, Driver, User, Car, and Ride tables.



CampusRide

Smart campus ride-sharing platform

M

Miray Murat

Admin

Passenger

Rides

All Rides

Ride Id	Passenger	Driver	Start Destination	End Destination	Ride Time	Cost	Rating	Plate Number
R001	İclal Akman	Badenur Turgut	METU NCC Dormitory	Nicosia City Center	2026-05-12 14:30:00	1200.00	5	33 ABC 123
R002	Miray Murat	Mert Yılmaz	METU NCC Library	Lefkoşa Bus Terminal	2026-05-13 09:00:00	900.00	4	06 MRT 789
R003	Zeynep Kaya	Deniz Şahin	Kalkanlı Gate	Girne City Center	2026-05-14 17:45:00	1500.00	2	35 DNZ 246
R004	Efe Demir	Badenur Turgut	Engineering Faculty	Güzelyurt Center	2026-05-15 11:15:00	750.00	5	33 ABC 123
R005	Elif Arslan	Efe Demir	Main Cafeteria	Nicosia City Center	2026-05-16 18:00:00	1100.00	4	34 EFE 456

My Passenger Rides

Ride	Driver	Start	End	Ride Time	Cost	Rating	Car	Actions
R002	Mert	METU NCC	Lefkoşa Bus	2026-05-13	900.00	4	06 MRT	Update Rating

```

if "Passenger" in roles:
rows = query_all("""
SELECT r.ride_ID AS ride_id,
du.name || ' ' || du.surname AS driver,
r.start_destination,
r.end_destination,
p.date_time AS ride_time,
r.cost,
r.rating,
r.plate_number,
(p.date_time <= CURRENT_TIMESTAMP) AS can_rate
FROM ride r
JOIN offer o ON r.offer_ID = o.offer_ID
JOIN post p ON o.post_ID = p.post_ID
JOIN "User" du ON o.driver_ID = du.user_ID
WHERE p.passenger_ID = %s
""", (u["user_id"],))
rate_action = f'<a class="btn primary" href="/ride/rate/{esc(r["ride_id"])}">{rate_label}'

```

6. Rate Ride Transaction

The Rate Ride operation is different from report submission. Rating is used only to evaluate the driver after the ride time has started. The passenger can give a rating between 1 and 5. This updates Ride.rating and then recalculates Driver.rate according to the average ratings of completed rides.

Database operation type: UPDATE

Tables used:

Ride, Driver, Offer


CampusRide
 Smart campus ride-sharing platform

M Miray Murat
 Admin Passenger

Ride	Driver	Start	End	Ride Time	Cost	Rating	Car	Actions
R002	Mert Yılmaz	METU NCC Library	Lefkoşa Bus Terminal	2026-05-13 09:00:00	900.00	4	06 MRT 789	Update Rating Report This Ride
R006	Badenur Turgut	Metu NCC 2nd Dorm	Girne Harbor	2026-06-06 21:25:00	1000.00		33 ABC 123	Rate available after ride time Report This Ride


CampusRide
 Smart campus ride-sharing platform

M Miray Murat
 Admin Passenger

[Home](#)
[Profile](#)
[Rides](#)
[Report](#)
[Create Post](#)
[Received Offers](#)
[Admin Panel](#)
[Logout](#)

Rate Ride

Rate the driver after the ride time has started. Your rating updates the ride and refreshes the driver's average rate.

Rating

5 - Excellent

[Save Rating](#)

```

execute("""
UPDATE driver d
SET rate = sub.avg_rating
FROM (
SELECT o.driver_ID, ROUND(AVG(r.rating)::NUMERIC, 2) AS avg_rating
FROM offer o
JOIN ride r ON r.offer_ID = o.offer_ID
WHERE r.rating IS NOT NULL
GROUP BY o.driver_ID

```

```

) sub
WHERE d.user_ID = sub.driver_ID
AND d.rate IS DISTINCT FROM sub.avg_rating;
))))

```

7. Report Transaction

The Report operation is used for problem reporting, not rating. A user can submit a report by entering a text explanation. If the report is about a specific ride, ride_ID is stored. If the report is a general platform issue, ride_ID can be NULL and report_type is set to General.

Database operation type: INSERT

Table used:

Report

Choose whether your report is about a specific ride or a general application issue.

Report a Ride

Use this option if the problem is related to a specific ride. Select the correct ride below and submit your explanation.

Report App Issue

Use this option if the problem is about the application, interface, login, data display, or another general platform issue.

[Report App Issue](#)

My Related Rides

Ride	Route	Ride Time	Passenger	Driver	Cost	Rating	Action
R006	Metu NCC 2nd Dorm → Girne Harbor	2026-06-06 21:25:00	Miray Murat	Badenur Turgut	1000.00	5	Report This Ride
R002	METU NCC Library → Lefkoşa Bus Terminal	2026-05-13 09:00:00	Miray Murat	Mert Yilmaz	900.00	4	Report This Ride

```

SELECT DISTINCT
r.ride_ID AS ride_id,
r.start_destination,
r.end_destination,
p.date_time AS ride_time,
pu.name || ' ' || pu.surname AS passenger,
du.name || ' ' || du.surname AS driver
FROM ride r
JOIN offer o ON r.offer_ID = o.offer_ID
JOIN post p ON o.post_ID = p.post_ID
JOIN "User" pu ON p.passenger_ID = pu.user_ID
JOIN "User" du ON o.driver_ID = du.user_ID
WHERE p.passenger_ID = %s OR o.driver_ID = %s

```

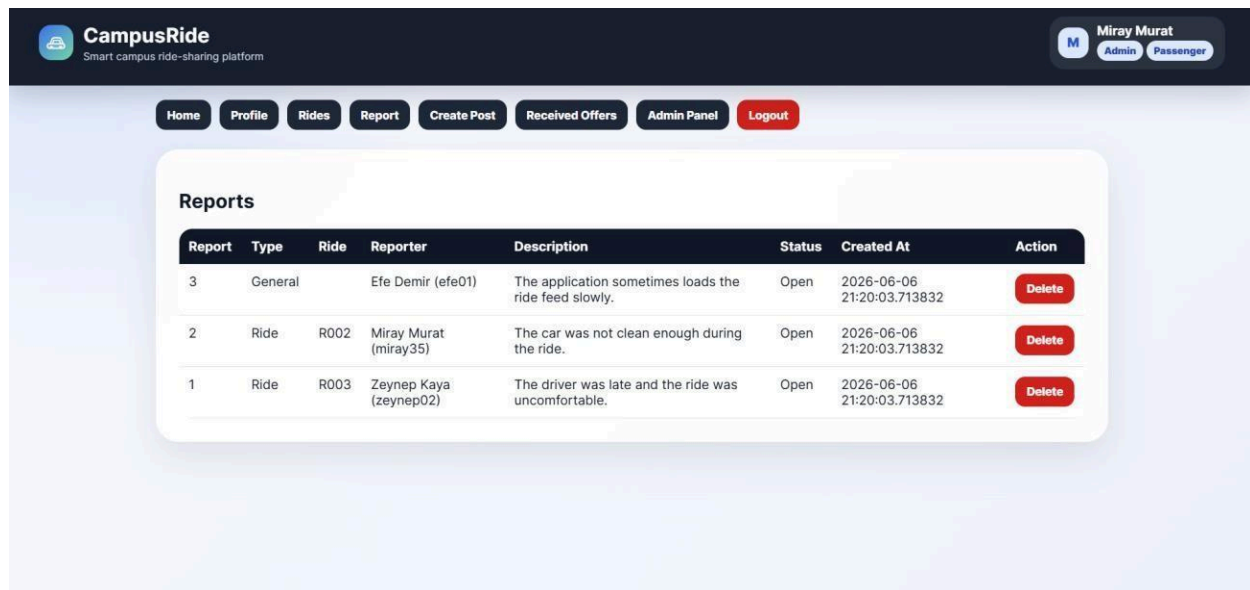
8. Admin Report Delete Transaction

Admins can view submitted reports and delete reviewed report records. This demonstrates a DELETE operation through the web interface.

Database operation type: DELETE

Table used:

Report



This screenshot shows the Admin Reports page. Admins can view both ride-specific reports and general application issue reports. The table includes report ID, report type, ride ID, reporter, report description, status, and creation time. Admins can delete reviewed reports from this page.

```
1465
1466 @app.route("/admin")
1467 @login_required
1468 @role_required("admin")
1469 def admin_panel():
1470     u = current_user()
1471     perms = "".join(f'<span class="pill">{esc(p)}</span>' for p in permissions_of(u["user_id"]))
1472     content = f'<h2>Admin Panel</h2><div class="card"><h3>Admin Permissions</h3><p>{perms}</p></div>{db_stat
1473     return layout("Admin Panel", content)
1474
1475
```

9. Admin Panel Data Queries

The Admin Panel provides several SELECT operations. Admins can list users, admins, permissions, drivers, reports, and rides. These pages demonstrate data retrieval from multiple relations.

Database operation type: SELECT

Tables used:

User, Admin, AdminPermission, Driver, Car, Ride, Report

CampusRide						
Smart campus ride-sharing platform						
Miray Murat						
Admin Passenger						
All Users						
User Id	Username	Name	Email	Phone	Gender	Action
1	miray35	Miray Murat	e252654@metu.edu.tr	05514404885	Female	Current admin
2	badenur48	Badenur Turgut	e252673@metu.edu.tr	05324567891	Female	Delete User
3	iclal26	İclal Akman	e258460@metu.edu.tr	05431234567	Female	Delete User
4	efe01	Efe Demir	e252701@metu.edu.tr	05551234567	Male	Delete User
5	zeynep02	Zeynep Kaya	e252702@metu.edu.tr	05339876543	Female	Delete User
6	mert03	Mert Yılmaz	e252704@metu.edu.tr	05445556677	Male	Delete User
7	deniz04	Deniz Şahin	e252706@metu.edu.tr	05321112233	Male	Delete User
8	elif05	Elif Arslan	e252707@metu.edu.tr	05553334455	Female	Delete User

```
rows = query_all(
SELECT user_ID AS user_id,user_name,name,surname,email,phone_number,gender FROM
"User" ORDER BY user_ID;
)
```

This screenshot shows the Admin Users page. Admins can list all registered users and delete user accounts when necessary. Deleting a user is an admin-level DELETE operation. The application handles related records carefully to prevent foreign key errors.

```
C: > Users > miroy > Downloads > campusride_professional_app_v12.py > admin_delete_user
1527 def admin_delete_user(user_id):
1563     WHERE driver_ID = %s
1564     OR post_ID IN (SELECT post_ID FROM post WHERE passenger_ID = %s)
1565 );
1566 """
1567 (user_id, user_id),
1568 )
1569
1570 cur.execute(
1571     """
1572     DELETE FROM offer
1573     WHERE driver_ID = %s
1574     OR post_ID IN (SELECT post_ID FROM post WHERE passenger_ID = %s);
1575     """
1576     (user_id, user_id),
1577 )
1578
1579 cur.execute("DELETE FROM post WHERE passenger_ID=%s;", (user_id,))
1580 cur.execute("DELETE FROM report WHERE user_ID=%s;", (user_id,))
1581 cur.execute("DELETE FROM car WHERE driver_ID=%s;", (user_id,))
1582 cur.execute("DELETE FROM adminpermission WHERE user_ID=%s;", (user_id,))
1583 cur.execute("DELETE FROM admin WHERE user_ID=%s;", (user_id,))
1584 cur.execute("DELETE FROM passenger WHERE user_ID=%s;", (user_id,))
1585 cur.execute("DELETE FROM driver WHERE user_ID=%s;", (user_id,))
1586 cur.execute("DELETE FROM \"User\" WHERE user_ID=%s;", (user_id,))
1587 affected = cur.rowcount
1588
1589 conn.commit()
1590 cur.close()
1591
1592 flash(f"User deleted successfully. Affected User rows: {affected}.")
1593
```